



**Documento de Requisitos
Funcionales:
Core de Cuentas**

Entidad: Banco BanQuito

Versión: 2.0

Fecha: Mayo 2026

Contenidos

1. Introducción	3
2. Conceptos Bancarios del Core	3
2.1 Registro Contable y Partida Doble	3
2.2 Plan de Cuentas Simplificado: Banco BanQuito (Evolución V2)	5
2.3 Operativa de Canales Satélites (Ventanilla y Banca Web)	6
2.4 Ciclo de Vida transaccional (Data Caliente vs. Data Fría)	7
3. Alcance del Sistema (Fase 2 - Evolución Financiera y Satélites)	8
4. Diagrama de Contexto	9
4.1. Consideraciones y Datos de Configuración (Entorno Ficticio).....	10
5. Requisitos Funcionales.....	10
Anexo.....	15
Anexo 1: Ejemplo de Modelo de BD para soporte de la contabilidad.....	15
Anexo 2: Particionamiento de Tablas en PostgreSQL	16

1. Introducción

El Core de Cuentas es el sistema maestro del Banco BanQuito encargado de la administración del ciclo de vida de los productos de depósito. Su función principal es mantener la integridad absoluta de los saldos, gestionar los estados de las cuentas y procesar de manera atómica las transacciones de débito y crédito solicitadas por los canales del banco y, fundamentalmente, por el Switch de Pagos Masivos.

En esta segunda evolución (Fase 2), el Core transforma radicalmente su arquitectura, abandonando el modelo monolítico centralizado para adoptar un enfoque de **Microservicios**. Esta transición distribuye el sistema en dominios independientes (Transaccional y Contable), garantizando mayor resiliencia. Asimismo, el sistema amplía su responsabilidad financiera al incorporar el registro contable de partida doble y la automatización de los procesos de **Fin de Día (EOD - End of Day)**, asegurando el cuadro corporativo y permitiendo la integración de nuevos canales satélites como Ventanilla y Banca Web Personas.

2. Conceptos Bancarios del Core

Esta sección define los términos de negocio aplicados a este sistema.

2.1 Registro Contable y Partida Doble

En esta fase de evolución del Core, el sistema deja de tratar a las cuentas como simples variables numéricas que suben o bajan, y adopta el modelo financiero universal. Esta sección define los términos contables que gobernarán el motor de transacciones:

- **Contabilidad de Partida Doble:** Principio fundamental que establece que todo movimiento financiero afecta obligatoriamente a, por lo menos, dos cuentas contables. No existe un débito sin un crédito equivalente. Esto garantiza que la ecuación contable fundamental del banco ($\text{Activos} = \text{Pasivos} + \text{Patrimonio}$) se mantenga siempre en equilibrio.
- **Catálogo de Cuentas (Plan Único de Cuentas):** Es la estructura jerárquica y codificada de todas las cuentas financieras que maneja la institución. Permite clasificar si un dinero le pertenece al banco, si se lo debe a un cliente, o si es una ganancia.
- **Naturaleza de las Cuentas Bancarias (Paradigma Invertido):** Para los ingenieros de software, es vital entender que la contabilidad del banco funciona "al revés" de la contabilidad personal del cliente:
 - **Cuentas de Pasivo (Obligaciones):** El dinero depositado en las cuentas de Ahorro y Corriente de los clientes **NO es dinero del banco**, es una deuda que el banco tiene con el cliente. Por lo tanto, cuando un cliente

deposita dinero, su saldo aumenta mediante un movimiento de **Crédito** (aumenta el pasivo del banco). Cuando retira, se aplica un **Débito**.

- **Cuentas de Activo (Bienes y Derechos):** Representa el dinero físico en las bóvedas, el encaje en el Banco Central o los préstamos otorgados.
- **Cuentas de Ingresos (Ganancias):** Registran el dinero que el banco gana por su operación, como el cobro de la comisión del Switch de Pagos Masivos. Aumentan mediante movimientos de **Crédito**.
- **Cuentas de Pasivo Transitorio (Impuestos):** Cuentas puente donde el banco retiene dinero de terceros que luego debe entregar al Estado (Ej. IVA retenido por comisiones).
- **Asiento Contable (Journal Entry):** Es el registro técnico, inmutable y formal de una transacción en el sistema. Un asiento válido debe tener un UUID, una fecha, una glosa (descripción) y al menos dos líneas de detalle ("patas" del asiento) cuya suma de débitos sea exactamente igual a la suma de créditos (Principio de suma cero).
- **Libro Mayor (General Ledger - GL):** Es el repositorio central o "fuente de la verdad" a nivel corporativo. Mientras que la tabla de transacciones de clientes muestra el detalle individual, el Libro Mayor consolida los saldos globales del banco.
- **Proceso EOD (End of Day / Cierre Diario):** Proceso *batch* nocturno donde el Core Bancario concilia todas las operaciones del día, verificando que los débitos y créditos totales del ecosistema cuadren a la perfección antes de aperturar la fecha contable del día siguiente.
- **Balance de Comprobación (Trial Balance):** Reporte contable fundamental que lista los saldos acumulados de todas las cuentas del catálogo en un momento dado. Su objetivo es verificar matemáticamente que la suma de todos los saldos deudores sea exactamente igual a la suma de todos los saldos acreedores, demostrando la integridad del motor financiero.
- **Fecha Real (Timestamp) vs. Fecha Contable:** La "Fecha Real" es el momento físico exacto en que ocurre la transacción en el servidor. La "Fecha Contable" es la fecha lógica bajo la cual se registrará el movimiento en los libros del banco, la cual dicta en qué proceso EOD se incluirá la transacción.
- **Horario de Corte (Cut-off Time):** Momento exacto del día (Ej. 20:00 horas) en el que el banco cierra su jornada operativa actual. Cualquier transacción que ingrese al sistema después de esta hora de corte, se procesará automáticamente con la Fecha Contable del siguiente día hábil (T+1).

Ejemplo de Asiento Contable - Cobro de Comisión por Pagos Masivos:

- **Débito:** \$1.15 a la Cuenta Corriente del Cliente Empresa (Disminuye el Pasivo del banco).

- **Crédito:** \$1.00 a la Cuenta de INGRESOS_SERVICIOS_MASIVOS (Aumenta la ganancia del banco).
- **Crédito:** \$0.15 a la Cuenta de PASIVOS_IVA_RETENIDO (Aumenta la obligación del banco con el Estado).
- *Regla de validación:* Total Débitos (\$1.15) = Total Créditos (\$1.15). El asiento es válido.

2.2 Plan de Cuentas Simplificado: Banco Banquito (Evolución V2)

Este catálogo utiliza la codificación estándar americana y de entes de control, reducida a tres clases fundamentales para este proyecto: **1 (Activos/Bienes del banco)**, **2 (Pasivos/Deudas del banco con clientes o el Estado)** y **4 (Ingresos/Ganancias del banco)**.

Código Contable	Nombre de la Cuenta	Tipo de Cuenta	Ubicación en la Base de Datos	Propósito en el Proyecto
1.0.0.00	ACTIVOS	Súper-clase	N/A (Estructural)	Agrupar los recursos y dinero propio del banco.
1.1.0.00	Disponibilidades	Grupo	N/A (Estructural)	Efectivo disponible de forma inmediata.
1.1.0.01	Banco Central / Cámara de Compensación	Detalle (Auxiliar)	INSTITUTIONAL_ACCO UNT	Cuenta puente donde se liquida el dinero neto que se envía o recibe de otros bancos (Esencial para la V2 del Switch).
1.1.0.02	Bóveda Central / Efectivo en Caja	Detalle (Auxiliar)	INSTITUTIONAL_ACCO UNT	Almacena el dinero físico y la liquidez del banco. Permite simular el asiento de apertura del banco (inyectar capital inicial); es la cuenta que reduce su saldo cuando un cliente retira efectivo de un cajero automático o ventanilla.
2.0.0.00	PASIVOS	Súper-clase	N/A (Estructural)	Agrupar todo el dinero que el banco debe a terceros.

Código Contable	Nombre de la Cuenta	Tipo de Cuenta	Ubicación en la Base de Datos	Propósito en el Proyecto
2.1.0.00	Obligaciones con el Público	Grupo	N/A (Estructural)	Depósitos de los clientes del banco.
2.1.0.01	Cuentas de Ahorros Clientes	Detalle (Sub-mayor)	ACCOUNT (SUPER_TYPE 'AHORROS') =	Consolida el saldo contable de todas las cuentas de ahorros de personas y empresas.
2.1.0.02	Cuentas Corrientes Clientes	Detalle (Sub-mayor)	ACCOUNT (SUPER_TYPE 'CORRIENTE') =	Consolida el saldo contable de todas las cuentas corrientes de las empresas emisoras.
2.2.0.00	Retenciones e Impuestos	Grupo	N/A (Estructural)	Dinero cobrado que no es del banco; se guarda para el Estado.
2.2.0.01	IVA Retenido por Servicios	Detalle (Auxiliar)	INSTITUTIONAL_ACCOUNT	Cuenta donde se acumula el 15% de IVA cobrado en las comisiones (Mapea a PASIVOS_IVA_RETENIDO).
4.0.0.00	INGRESOS	Súper-clase	N/A (Estructural)	Agrupar las ganancias reales y operativas del banco.
4.1.0.00	Ingresos por Servicios	Grupo	N/A (Estructural)	Facturación por servicios bancarios.
4.1.0.01	Comisiones por Pagos Masivos	Detalle (Auxiliar)	INSTITUTIONAL_ACCOUNT	Cuenta de rentabilidad del banco donde va el neto de la comisión cobrada (Mapea a INGRESOS_SERVICIOS_MASIVOS).

2.3 Operativa de Canales Satélites (Ventanilla y Banca Web)

Para que el Core procese correctamente las peticiones de los nuevos canales, es necesario entender la naturaleza transaccional de cada uno:

- **Transacción de Ventanilla (Manejo de Efectivo):** Son operaciones físicas (depósitos o retiros) ejecutadas por un cajero humano en una sucursal.

Contablemente, estas transacciones siempre afectan a la cuenta 1.1.0.02 - Bóveda Central / Efectivo en Caja. Por ejemplo, un depósito en efectivo implica un débito a la Bóveda (aumenta el dinero físico del banco) y un crédito a la cuenta de ahorros (aumenta la deuda del banco con el cliente).

- **Transferencia Interna (Peer-to-Peer / P2P):** Operación digital y de autoservicio iniciada desde la Banca Web Personas. A diferencia del Switch de Pagos (que es de "uno a muchos"), la transferencia P2P es de "uno a uno" dentro del mismo ecosistema. Contablemente, es un traslado de saldos que afecta simultáneamente a dos cuentas de clientes (débito a la cuenta origen, crédito a la cuenta destino), sin tocar el efectivo del banco ni salir a la Cámara de Compensación.

2.4 Ciclo de Vida transaccional (Data Caliente vs. Data Fría)

Debido a la alta transaccionalidad inyectada por el Switch de Pagos y la Banca Web, el Core debe gestionar el almacenamiento histórico para evitar la degradación del motor de base de datos:

- **Data Caliente (Datos Activos):** Movimientos financieros recientes (por lo general de los últimos 3 a 6 meses) que los clientes consultan constantemente al abrir su aplicación móvil o Banca Web. Requieren índices optimizados y deben estar disponibles en milisegundos.
- **Data Fría (Datos Históricos / Archivados):** Transacciones antiguas que ya no son de consulta operativa diaria, pero que el banco tiene la obligación legal de conservar para auditorías o regulaciones (compliance). Esta información justifica el diseño de particionamiento de bases de datos, permitiendo separar físicamente el historial sin alterar la lógica de la aplicación.

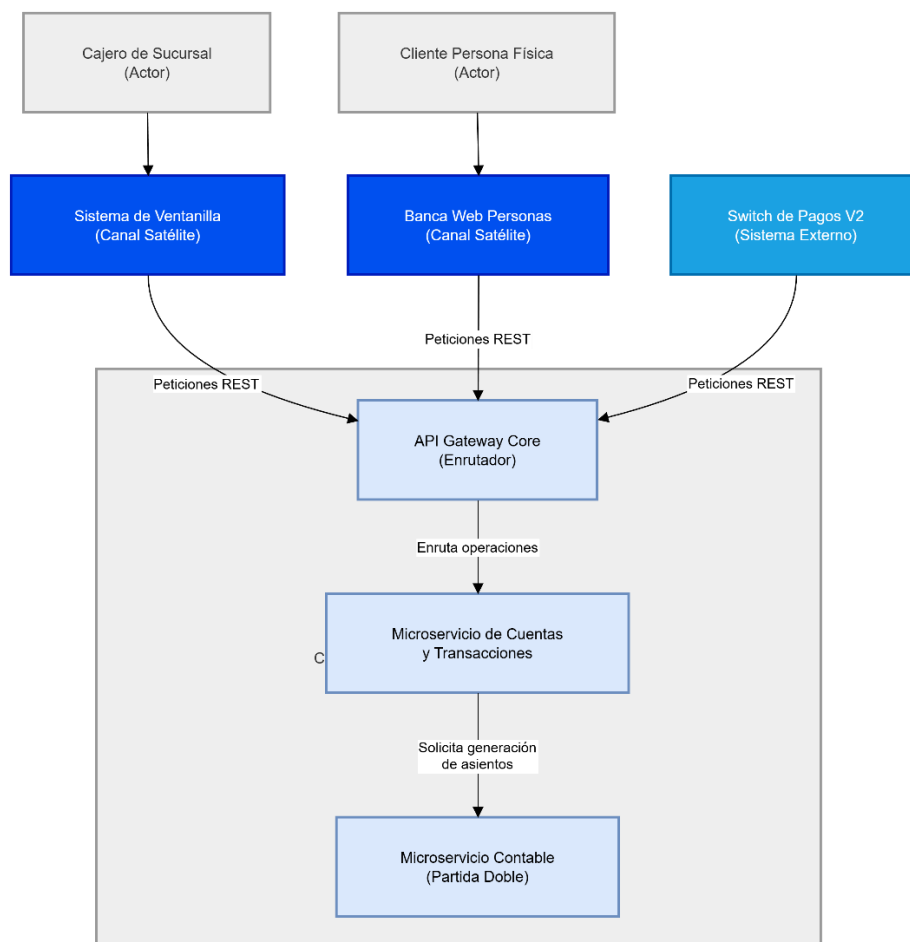
3. Alcance del Sistema (Fase 2 - Evolución Financiera y Satélites)

El alcance de esta segunda entrega contempla la evolución del Core de Cuentas para convertirse en un motor financiero robusto y auditable, además de expandir su integración hacia nuevos canales de atención.

Están DENTRO del alcance de esta fase:

- **Motor Contable de Partida Doble:** Implementación del Plan de Cuentas y generación estricta de asientos contables (suma cero) por cada transacción financiera ejecutada en el sistema. El Core es el responsable absoluto de afectar las cuentas del cliente y las cuentas institucionales del banco.
- **Gestión de Canales Satélites:** Exposición de servicios web (APIs) para soportar la operatividad del sistema de Ventanilla (depósitos y retiros en efectivo) y del portal de Banca Web Personas (transferencias directas P2P y consultas de saldos).
- **Integración Pasiva con el Switch de Pagos:** El Core actuará como proveedor de servicios para el Switch de Pagos Masivos V2, recibiendo y ejecutando instrucciones de liquidación de nóminas (*On-Us*) y cálculos de comisiones (con sus respectivos impuestos).
- **Procesamiento de Fin de Día (EOD) y Cierre Contable:** Automatización del proceso de transición de la jornada bancaria. Incluye la gestión dinámica de la fecha contable (horarios de corte) y la generación estructurada de los reportes financieros que demuestren el cuadro de la institución (Balance de Comprobación).
- **Desacoplamiento Arquitectónico (Microservicios):** El Core abandonará su estructura monolítica para dividirse en, al menos, dos contextos delimitados independientes: un **Microservicio Transaccional (Cuentas y Saldos)** y un **Microservicio Contable (Libro Mayor)**. Cada uno deberá poseer su propia base de datos aislada (Database per Service) y exponer sus propios endpoints.
- **Particionamiento Histórico de Base de Datos:** Implementación de estrategias de particionamiento declarativo (por rangos de fechas) en el motor de base de datos PostgreSQL para la tabla de transacciones de cuentas, separando físicamente la data caliente de la data fría sin afectar la lógica de la aplicación.

4. Diagrama de Contexto



Descripción de los Componentes del Ecosistema

1. Cajero de Sucursal (Actor) Personal operativo del banco que atiende a los clientes físicamente. Su rol es interactuar con el sistema de Ventanilla para legalizar el ingreso o salida de dinero en efectivo (Data Física).

2. Cliente Persona Física (Actor) Usuario final del banco (cuentahabiente) que utiliza la Banca Web para gestionar su dinero personal de manera autónoma, realizando consultas de saldos y transferencias hacia terceros sin intervención del personal del banco.

3. Sistema de Ventanilla (Canal Satélite) Interfaz gráfica utilizada en las sucursales físicas. Consume las APIs del Core para ejecutar transacciones de depósito y retiro. Debe notificar al Core sobre el uso de la cuenta contable de la bóveda (1.1.0.02) en cada operación de efectivo.

4. Banca Web Personas (Canal Satélite) Portal de autoservicio para clientes individuales. Permite la ejecución de transferencias P2P (*Peer-to-Peer*) directas. Al igual que la Ventanilla, es un consumidor de las APIs expuestas por el Core.

5. Core Bancario V2 (El Sistema Central) Es el motor principal de registro y el único custodio de la "fuente de la verdad". En esta versión, procesa de forma síncrona las solicitudes de todos los canales. Tras validar los fondos, afecta los saldos, inyecta el registro en la tabla transaccional particionada (ACCOUNT_TRANSACTION) y genera de forma invisible el registro en el microservicio contable.

6. Switch de Pagos Masivos V2 (Sistema Externo/Distribuidor) Desde la perspectiva del Core, el Switch es un canal más, pero de alto volumen. El Switch interactúa con el Core exclusivamente para liquidar de forma atómica los pagos locales (*On-Us*) hacia las cuentas de los beneficiarios y para solicitar el débito de las comisiones en las cuentas de las empresas emisoras.

4.1. Consideraciones y Datos de Configuración (Entorno Ficticio)

Para asegurar que el Banco BanQuito sea funcional para las pruebas y la implementación del Switch, el Core se inicializará con la siguiente volumetría y reglas:

- **Clientes Totales:** 9000 Clientes Individuales y 500 Clientes Corporativos (Empresas).
- **Cuentas Totales:** 15000 cuentas activas.
- **Distribución de Cuentas:** * Cada cliente individual tendrá al menos una cuenta.
 - El 20% de los clientes individuales tendrán dos cuentas (Ahorros y Corriente).
 - Cada empresa (Corporativo) tendrá al menos 3 cuentas para gestionar diferentes flujos de caja (Operativa, Nómina, Impuestos).
- **Sucursales:** El banco operará con 5 sucursales distribuidas geográficamente (Norte, Sur, Centro, Valles, Digital).

5. Requisitos Funcionales

RF-01: Motor Contable y Sincronismo Distribuido

El sistema registrará la partida doble operando bajo un esquema de microservicios con bases de datos independientes.

- El Microservicio Contable es el único custodio del Plan de Cuentas y de las tablas de asientos (JOURNAL_ENTRY).
- Regla de Negocio y Arquitectura: Una vez que el Microservicio de Cuentas afecta exitosamente el saldo de un cliente, debe consumir de manera síncrona (vía API

REST o gRPC) al Microservicio Contable para solicitar la generación del asiento respectivo.

- Si el Microservicio Contable rechaza la petición (ej. el asiento no cumple la regla de Suma Cero) o si existe una falla de red (*Timeout*), el Microservicio de Cuentas está obligado a ejecutar de inmediato una transacción compensatoria (reverso) en su propia base de datos para anular el movimiento del cliente y mantener la consistencia del ecosistema.

RF-02: Gestión Transaccional de Ventanilla

El sistema debe exponer servicios web síncronos (APIs) para soportar las operaciones de los cajeros en sucursales físicas.

- Se debe implementar el endpoint para "Depósito en Efectivo".
- Se debe implementar el endpoint para "Retiro en Efectivo".
- **Regla de Negocio:** Toda transacción originada por este canal satélite es de naturaleza física, por lo tanto, el motor contable debe incluir obligatoriamente una afectación a la cuenta institucional de liquidez física (Bóveda Central / Efectivo en Caja) como contrapartida al movimiento de la cuenta del cliente.

RF-03: Operatividad de Banca Web Personas

El sistema debe exponer servicios web síncronos (APIs) para habilitar el autoservicio digital de los cuentahabientes individuales.

- Se debe exponer un servicio de consulta de saldo actualizado y consolidado.
- Se debe exponer un servicio para la ejecución de "Transferencias Directas (P2P)", permitiendo el traslado de fondos entre dos cuentas pertenecientes al Banco BanQuito.
- **Regla de Negocio:** El Core debe validar de forma estricta que la cuenta origen tenga saldo disponible suficiente y que la cuenta destino se encuentre en estado "Activa" antes de ejecutar el traslado de fondos.

RF-04: Liquidación y Cobro Delegado (Integración Switch)

El Core actuará como procesador pasivo de las instrucciones enviadas por el Switch de Pagos Masivos V2, exponiendo endpoints transaccionales específicos.

- Se debe habilitar un servicio para la recepción y acreditación de lotes de pagos internos (*On-Us*).
- Se debe habilitar un servicio para el débito corporativo de nóminas a la cuenta matriz de la empresa.

- **Regla de Negocio:** El endpoint de débito corporativo recibirá en su *payload* el Monto Total a dispersar y el Monto Total de la Comisión cobrada por el Switch. El Core es el único responsable de desglosar este último valor para calcular los impuestos de ley (Ej. IVA) y repartir el dinero entre las cuentas contables de Ingresos del Banco y de Pasivos Transitorios (Impuestos Retenidos).

RF-05: Particionamiento de Datos Transaccionales

Para garantizar el rendimiento en consultas concurrentes y preparar el sistema para auditorías históricas, el almacenamiento de las transacciones de las cuentas debe ser optimizado a nivel de motor de base de datos.

- La bitácora de movimientos debe dividir físicamente la "Data Caliente" de la "Data Fría".
- **Regla de Arquitectura:** La tabla principal de movimientos (ACCOUNT_TRANSACTION) debe implementarse utilizando el particionamiento declarativo nativo de PostgreSQL (Declarative Partitioning). El criterio de partición debe ser por rangos de fechas (Range Partitioning) utilizando la columna de la fecha de la transacción, generando subtablas físicas de forma mensual.

RF-06: Módulo Satélite de Ventanilla (Teller)

Se debe desarrollar una interfaz gráfica de usuario (Frontend/Cliente Web) independiente, diseñada para el uso exclusivo del personal de sucursales (cajeros).

- **Búsqueda y Validación:** La interfaz debe permitir al cajero buscar al cliente o la cuenta mediante su número de identificación o número de cuenta, mostrando el nombre del titular y el estado de la cuenta antes de proceder.
- **Gestión de Efectivo:** Debe incluir formularios específicos para procesar "Depósitos" y "Retiros", capturando el monto exacto de la operación.
- **Consumo de Servicios:** La interfaz debe capturar los datos ingresados por el cajero y orquestar la petición HTTP hacia las APIs síncronas expuestas por el Core Bancario (detalladas en el RF-02).
- **Comprobantes:** Tras recibir una respuesta HTTP 200 OK del Core confirmando que la transacción y la contabilidad se registraron exitosamente, la Ventanilla debe generar un comprobante digital (recibo) con el número de transacción para el cliente.

RF-07: Módulo Satélite de Banca Web Personas

Se debe desarrollar un portal web de autoservicio (Frontend) orientado al cliente final (cuentahabiente).

- **Panel Consolidado (Dashboard):** Al iniciar sesión, la aplicación web debe consumir de forma automática el API de saldos del Core (RF-03) y renderizar en pantalla el balance actual y disponible de las cuentas del cliente.
- **Formulario de Transferencias P2P:** Debe proveer una pantalla intuitiva donde el cliente seleccione su cuenta de origen (combo box validando que tenga saldo), ingrese el número de la cuenta destino y el monto a transferir.
- **Validación de Destinatario:** Antes de confirmar la transferencia, el portal debe consultar al Core para mostrar el nombre del titular de la cuenta destino, asegurando que el cliente verifique a quién le está enviando el dinero.
- **Manejo de Excepciones:** La interfaz debe capturar los códigos de error que devuelva el Core (por ejemplo: 400 Bad Request por fondos insuficientes o cuenta inactiva) y traducirlos a mensajes de error amigables y comprensibles para el usuario final, evitando mostrar trazas del sistema (stack traces).

RF-08: Gestión de Fecha Contable y Horario de Corte

El Microservicio de Cuentas debe desacoplar la fecha física de la fecha de negocio para garantizar la integridad del cierre diario.

- Toda transacción (depósito, retiro, transferencia o pago) debe registrarse obligatoriamente con dos campos de tiempo distintos: TRANSACTION_TIMESTAMP (fecha y hora del servidor) y ACCOUNTING_DATE (fecha contable).
- **Regla de Negocio:** El sistema debe mantener una variable global o paramétrica de "Fecha Contable Activa". Si una transacción ingresa después del horario de corte configurado para la ejecución del EOD, el sistema procesará la operación, afectará el saldo, pero su ACCOUNTING_DATE será marcada con la fecha del siguiente día hábil.

RF-09: Proceso de Fin de Día (EOD) y Balance de Comprobación

El Microservicio Contable debe exponer un proceso (API o Job programado) que ejecute el Cierre Operativo Diario y genere la reportería formal.

- **Transición de Jornada:** Al disparar el proceso EOD, el sistema debe cambiar automáticamente la "Fecha Contable Activa" al día siguiente, cerrando la admisión de nuevos registros para la fecha actual.

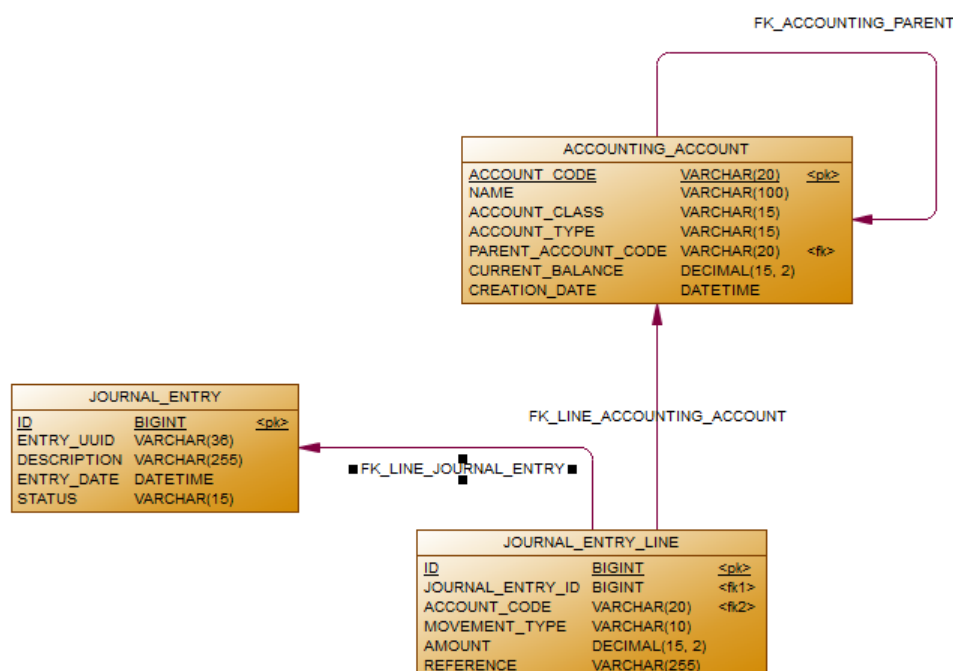
- **Reporte Estructurado:** El sistema debe generar el Balance de Comprobación consolidando los saldos finales de las cuentas ESTRUCTURALES del catálogo (Activos, Pasivos, Patrimonio, Ingresos y Gastos).
- **Formato de Salida:** El reporte debe ser exportado obligatoriamente en formato **CSV** (Valores separados por comas) y almacenado en una ruta segura del servidor, conteniendo las columnas: Código de Cuenta, Nombre de Cuenta, Saldo Deudor, Saldo Acreedor.
- **Regla de Negocio (Validación de Cuadre):** La última fila del reporte CSV debe contener la totalización. El proceso EOD no podrá darse por finalizado exitosamente si la sumatoria de saldos deudores difiere de la sumatoria de saldos acreedores.

Anexo

Anexo 1: Ejemplo de Modelo de BD para soporte de la contabilidad.

Importante: Este es solo un ejemplo didáctico. En la implementación se debe definir un modelo completo.

Revise la justificación del modelo propuesto; sin embargo, es muy posible que falte información.



Justificación del diseño:

- La Relación Recursiva (FK_ACCOUNTING_PARENT):** Permite modelar el árbol contable. Por ejemplo, la cuenta 1.1.0.01 (Banco Central) tendrá como PARENT_ACCOUNT_CODE a la cuenta 1.1.0.00 (Disponibilidades), y esta a su vez apuntará a 1.0.0.00 (Activos).
- La Bandera ACCOUNT_TYPE (Estructural vs. Detalle):** El sistema **nunca** debe permitir registrar un asiento contable en una cuenta "Estructural" (como 1.0.0.00 - ACTIVOS), ya que estas solo sirven para acumular los saldos de sus hijos. Los asientos solo se pueden hacer en cuentas de tipo DETALLE (nodos hoja).
- La Relación entre el Core Transaccional y la Contabilidad:** La columna ENTRY_UUID en JOURNAL_ENTRY es el puente de trazabilidad. Cuando el Switch procese una transacción y el Core genere el movimiento en la cuenta bancaria del cliente, el ID único de esa transacción (TRANSACTION_UUID) se guardará también aquí. Si un auditor pregunta de dónde salió el asiento contable número 45, el UUID le dirá exactamente qué transacción del cliente lo provocó.

Anexo 2: Particionamiento de Tablas en PostgreSQL

PostgreSQL soporta el particionamiento de tablas de forma nativa y con un rendimiento excepcional. De hecho, implementa lo que se conoce como *Declarative Partitioning* (Particionamiento Declarativo), el cual es ideal porque se maneja directamente mediante sentencias DDL de SQL estándar.

Para la evolución del Core a la Versión 2, la estrategia perfecta que se debe aplicar en la tabla de transacciones históricas (ACCOUNT_TRANSACTION) es el **Particionamiento por Rango (Range Partitioning)** utilizando la columna TRANSACTION_DATE.

Esto permitirá definir una estructura donde la tabla ACCOUNT_TRANSACTION actúa como una interfaz lógica, pero físicamente los datos se guardan en subtablas automáticas divididas por tiempo (por ejemplo, una partición por mes o por año: TRANSACTION_2026_01, TRANSACTION_2026_02).